Chapter 14
Scientific Software

Abstract In this chapter we present engineering libraries such as Computer Vision, CImg and FWTools. Geospatial data abstractions are becoming very important with the rise of location aware computing, and several open-source tools such as GDAL and PROJ4 are described in this chapter. Image processing, audio processing, and computational fluid dynamics (CFD) have been part of many engineering applications. More recently, molecular dynamics and simulation programs have also become heavy contenders for the compute time on grids. Molecular dynamics programs (GROMACS, NAMD) as well molecule viewers (JMol) are described in this chapter. Geographical Information Systems (GIS) including GRASS and QGIS are described in Section 14.9.1. Mechanical engineering, as well as use of mechanical CAD software in other disciplines can be accomplished using open-source tools such as QCAD. Solid modeling tools BRL-CAD are described in Section 14.11 as well as Blender 14.12.

Contents

14.1 Computer Vision with OpenCV **288**
14.2 CImg: C Image Processing Toolkit **289**
14.3 Binary Decision Diagram (bdd): CUDD Library **291**
14.4 FWTools: Open Source GIS **295**
14.5 GNU Image Manipulation Program **298**
14.6 Computational Fluid Dynamics using OpenFOAM **300**
14.7 Molecular Dynamics **303**
14.8 Audacity **306**
14.9 Geographical Information Systems **307**
14.10 QCAD : 2d CAD Tools **309**
14.11 BRL-CAD **311**
14.12 Blender **314**
14.13 Conclusion **315**

In previous chapters we have discussed the underlying operating system, user shell interaction, various application libraries, compilers and more. In this chapter we focus on integrated open-source software solutions for engineering problems. The domains of engineering include (i) image processing, (ii) audio processing,

S. Koranne, *Handbook of Open Source Tools*,
DOI 10.1007/978-1-4419-7719-9_14, © Springer Science+Business Media, LLC 2011

(iii) finite-element analysis, (iv) computational biology, (v) geographical information systems, and (vi) VLSI and electronics. This list is not exhaustive, but is representative of the many domains in which open-source software has been used with great positive effect. We first discuss engineering libraries which are used in many open-source software for engineering.

14.1 Computer Vision with OpenCV

OpenCV (Open Source Computer Vision) is a library for real time computer vision. OpenCV has the following modules in its source-code:

1. cxcore: Core functionality including basic structures, array operations, drawing functions, XML, clustering and utility system function. An example of using OpenCV core library is shown in Listing 14.1.

```
// \file cv_example.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of using cxCore
#include <iostream>           // Program IO
#include <cassert>            // assertions
#include <cv.h>               // OpenCV
#include <highgui.h>          // OpenCV GUI

static CvMemStorage *storage;
static CvRect gRectA;
static void *data = NULL;
static void InitializeData( ) {
  gRectA.x = 0, gRectA.y = 0;
  gRectA.width = 100, gRectA.height = 100;
  storage = cvCreateMemStorage( 0 );
  data = cvMemStorageAlloc( storage, 1024 );
}

static void PrintImageInformation(const IplImage* image ) {
  std::cout << "Image size   = " << image->nSize
      << "Num channels = " << image->nChannels
      << "Depth        = " << image->depth
      << std::endl;
}


int main( int argc, char* argv[] ) {
  if( argc != 3 ) {
    std::cerr << "Usage: ./ipl_example <file> <file>...\n";
    exit(1);
  }
  IplImage *image = cvLoadImage( argv[1] );
  assert( image && "Unable to load image ");
  PrintImageInformation( image );
  InitializeData();
  CvSize dsize; dsize.width = dsize.height = 100;
  //IplImage *dest = cvCreateImageHeader( dsize, 8, 3 );
  IplImage *dest = cvCreateImage( dsize, 8, 3 );
  CvSeq *contour = NULL;
  cvThreshold( image, image, 1, 255, CV_THRESH_BINARY );
  cvFindContours( image, storage, &contour,
      sizeof( CvContour ), CV_RETR_CCOMP,
      CV_CHAIN_APPROX_SIMPLE );
  CvScalar color = CV_RGB( 143, 100, 200 );
```

```
    for(; contour; contour = contour->h_next ) {
      cvDrawContours( dest, contour, color, color,
          -1, CV_FILLED, 8 );
    }
    cvSaveImage( argv[2], dest );
    cvReleaseImage( &image );
    cvReleaseImage( &dest );
    cvReleaseMemStorage( &storage );
    std::cout << std::endl;
    return (0);
}
```

Listing 14.1 OpenCV core functionality

2. cv : Image processing and computer vision: image filtering, geometric image transforms, histograms, feature detection, motion analysis and object tracking, planar subdivisions, object detection and camera calibration,
3. ml : Machine Learning: statistical models, Bayesian classifier, *k*-nearest neighbors, support vector machines (SVM), decision trees, boosting, random trees and neural networks.

An example of using OpenCV for image processing and display is shown in Listing 14.2.

```
// \file ipl_example.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of using OpenCV
#include <iostream>           // Program IO
#include <cstdlib>            // exit
#include <highgui.h>          // OpenCV GUI

// Simple program to display an image
int main( int argc, char* argv[] ) {
  if( argc != 2 ) {
    std::cerr << "Usage: ./ipl_example <file>...\n";
    exit(1);
  }
  IplImage *image = cvLoadImage( argv[1] );
  assert( image && "Unable to load image ");
  cvNamedWindow( "IPL_EXAMPLE", CV_WINDOW_AUTOSIZE );
  cvShowImage( "IMAGE", image );
  cvWaitKey( 0 ); // get user input
  cvReleaseImage( &image );
  cvDestroyWindow( "IPL_EXAMPLE" );
  std::cout << std::endl;
  return(0);
}
```

Listing 14.2 Displaying images using OpenCV

Compiling and running this program produces the image as shown in Figure 14.1.

14.2 CImg: C Image Processing Toolkit

The CIMG (C++ Template Image Processing Toolkit) defines classes and method to process image in C++ programs. It defines a single Image class which can represent multi-dimensional datasets upto 4-dimensions with templatized pixel types. It can

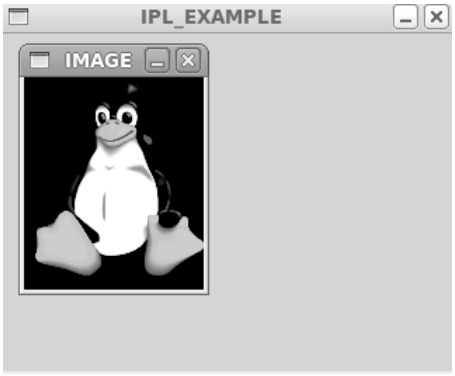


Fig. 14.1 OpenCV 'highgui' image display example

also handle image collections and *sequences*. CIMG is also very efficient and simple to use since only the single header file `CImg.h` needs to be included, and all functions and classes or CIMG are encapsulated in its own namespace. It is self-contained, however, it can be used alongwith other open-source packages such as libpng (see Section 11.2.1), libjpeg (see Section 11.2.1.2), FFTW3 (see Section 16.8) and LAPACK (see Section 16.3).

A small example to demonstrate CIMG Toolkit is shown in Listing 14.3.

```
// \file cimg_color.cpp
// \author Sandeep Koranne (C) 2010
// \description Example of using CImg toolkit
#include <cstdlib>            // for exit
#include <iostream>           // for program IO
#include <fstream>            // std::ofstream
#include <CImg.h>             // CIMG Toolkit
#include <vector>             // std::vector
#include <algorithm>          // std::copy
#include <iterator>           // ostream iterator
#include <cassert>            // assertion checking

using namespace cimg_library;

int main( int argc, char *argv [] ) {
  if( argc != 2 ) {
    std::cerr << "Usage: ./cimg_color <file>...\n";
    exit(1);
  }
  const CImg<unsigned char> image = CImg<>( argv[1] );
  const unsigned int W = image.width();
  const unsigned int H = image.height();
  const unsigned int D = std::min( W, H );
  std::assert( D > 0 );
  std::cout << "Image H = " << H  << "\t"
      << "Image W = " << W << std::endl;
  // calculate the color profile on line going from (0,0) -> (W,H)
  std::vector<unsigned int> RED( D );
  std::vector<unsigned int> GREEN( D );
  std::vector<unsigned int> BLUE( D );
  for( unsigned int i=0; i < std::min( W, H ); ++i ) {
    RED[i]   = image( i, i, 0 );
    GREEN[i] = image( i, i, 1 ); // RGB = (0,1,2)
    BLUE[i]  = image( i, i, 2 );
  }
  std::ofstream r_dat("red.dat"), g_dat("green.dat"), b_dat("blue.dat");
```

```
  std::copy( RED.begin(), RED.end(),
       std::ostream_iterator<unsigned int>( r_dat, "\n" ) );
  std::copy( GREEN.begin(), GREEN.end(),
       std::ostream_iterator<unsigned int>( g_dat, "\n" ) );
  std::copy( BLUE.begin(), BLUE.end(),
       std::ostream_iterator<unsigned int>( b_dat, "\n" ) );
  std::cout << std::endl;
  return (0);
}
```

Listing 14.3 Example of using CImg Toolkit

The program shown in Listing 14.3 calculates the color profile (RGB value) on the diagonal line joining (0,0) to min(W,H) of the given image. The use of C++ templates in CImg is shown as well.

14.3 Binary Decision Diagram (bdd): CUDD Library

Binary Decision Diagrams, are a very useful tool for modeling binary variables and systems. They are primarily used in VLSI CAD for logic synthesis and optimization. They can also be used to perform optimization and Boolean reductions in domains where the problem can be expressed as a Boolean decision problem.

CUDD is an acronym for the Colorado University Decision Diagram Package. It is an open-source C/C++ library for creating and managing BDDs as well as ZBDDs (zero-suppressed BDDs). CUDD implements an internal garbage-collector (see Section 7.3 for a discussion of garbage-collectors), thus the programmer must *reference* and *dereference* BDD nodes during the program computation. By design, CUDD uses an *unique table* to record BDDs; this ensures that identical functions map to the same BDD node and is used for canonical checking on functions for equivalence.

Binary Decision Variables are implemented as a binary tree, where each node has a *then* child, and an *else* child (which are traversed when we assign the value of this BDD node to true, or false, respectively). For each variable listed in the function we traverse the path, and the value of the function is the value of the leaf node we arrive at (true or false). However, for efficiency, the children nodes of a node can be complemented, which complements the value of our assignment. The CUDD library implements Reduced Ordered BDDs, and in this section we present examples of using CUDD to solve problems with BDDs. A short example is shown in Listing 14.4.

```
// \file bdd_example.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of using BDD with CUDD library
#include <iostream>           // program IO
#include <cassert>            // assertion checking
#include <cstdio>             // C stdio
#include <cudd.h>             // CUDD BDD library


static DdManager* gManager;   // global DD Manager
static DdNode**   gNodeArray; // pointer to nodes
```

```
static unsigned int numVars;  // number of variables

static void PrintStats(void) {
  std::cout
    << "CUDD Statistics"
    << "\nNum. vars: " << Cudd_ReadSize( gManager )
    << "\nNum. count:" << Cudd_ReadNodeCount( gManager )
    << "\nNum. order:" << Cudd_ReadReorderings( gManager )
    << "\nMemory    :" << Cudd_ReadMemoryInUse( gManager )
    << std::endl;
}

int main( int argc, char* argv[] ) {
  // print CUDD version information
  std::cout << "CUDD Version = ";
  Cudd_PrintVersion( stdout );
  numVars = 4; // simple example
  gManager = Cudd_Init( numVars, 0, CUDD_UNIQUE_SLOTS,
      CUDD_CACHE_SLOTS, 1024*1024 );
  assert( gManager && "Unable to create CUDD Manager" );

  DdNode *x0 = Cudd_bddIthVar( gManager, 0 ); // x0
  DdNode *x1 = Cudd_bddIthVar( gManager, 1 ); // x1
  DdNode *x0_and_x1 = Cudd_bddAnd( gManager, x0, x1 );
  Cudd_Ref( x0_and_x1 );
  DdNode *x0_and_bar_x1 =
    Cudd_bddAnd( gManager, x0, Cudd_Not( x1 ) );
  Cudd_Ref( x0_and_bar_x1 );
  DdNode *functionA = Cudd_bddAnd( gManager, x0_and_bar_x1,
           x0_and_x1 );
  DdNode *functionB = Cudd_bddOr( gManager, x0_and_bar_x1,
           x0_and_x1 );

  Cudd_Ref( functionA );
  Cudd_Ref( functionB );
  Cudd_Ref( x0_and_x1 );
  FILE* f = fopen("bdd.dot","w");
  char **inputNames = new char*[2];
  inputNames[0] = new char[3]; inputNames[0] = "x0";
  inputNames[1] = new char[3]; inputNames[0] = "x1";
  char **outputNames = new char*[3];
  outputNames[0] = new char[10]; outputNames[0] = "functionA";
  outputNames[1] = new char[10]; outputNames[1] = "functionB";
  outputNames[2] = new char[10]; outputNames[2] = "x0_and_x1";

  DdNode **outputs = new DdNode*[3];
  outputs[0] = functionA;
  outputs[1] = functionB;
  outputs[2] = x0_and_x1;
  Cudd_DumpDot( gManager, 3, outputs,
    inputNames, outputNames, f );
  PrintStats();
  std::cout << std::endl;
  return(0);
}
```

Listing 14.4 Using CUDD BDD Library

As shown in Listing 14.4 operations on BDDs are managed by the `DdManager` opaque object. To start the CUDD system we call the `Cudd_Init` function which sets up the memory space for the required number of variables.

Each `DdNode` represents a variable in the system. The data-structure for `DdNode` contains the variable index, reference count, a `next` pointer for the unique table, and a union type containing either the constant nodes or the BDD children.

The following functions operate on the BDD Node structure `DdNode`:

Table 14.1 CUDD DdNode Functions

Name	Description and Return value
`Cudd_IsConstant`	1 if node is leaf
`Cudd_T`	a pointer to "then" child
`Cudd_E`	a pointer to "else" child
`Cudd_IsComplement`	1 if node is complement
`Cudd_Regular`	1 if node is regular
`Cudd_V`	value of constant node

To build the example shown in Listing 14.4 we use the following SConstruct file:

```
CUDD="/home/skoranne/cudd-2.4.1/"
Program('bdd_example',['bdd_example.cpp'],
        CPPPATH=CUDD+"include",
        LIBPATH=[CUDD+"cudd",
                 CUDD+"st",
                 CUDD+"mtr",
                 CUDD+"epd",
                 CUDD+"util"],
        LIBS=['cudd','util','st','mtr','epd'])
```

Running `scons` gives:

```
$ scons
scons: Reading SConscript files ...
scons: done reading SConscript files.
scons: Building targets ...
g++ -o bdd_example.o -c
    -I/home/skoranne/cudd-2.4.1/include bdd_example.cpp
g++ -o bdd_example bdd_example.o
        -L/home/skoranne/cudd-2.4.1/cudd
        -L/home/skoranne/cudd-2.4.1/st
        -L/home/skoranne/cudd-2.4.1/mtr
        -L/home/skoranne/cudd-2.4.1/epd
        -L/home/skoranne/cudd-2.4.1/util
        -lcudd -lutil -lst -lmtr -lepd
scons: done building targets.

# Running the program
$ ./bdd_example
CUDD Version = 2.4.1
CUDD Statistics
Num. vars: 4
Num. count:5
Num. order:0
Memory    :4225732
```

Using the BDD function we can model the Boolean functions using combinations of AND and OR functions. We can also print the BDD collection to a DOT (see Section 19.4) file. The produced BDD collection is shown in Figure 14.2. It is instructive to see the relationship between the paths, complemented paths from variables in Figure 14.2, to the BDD construction code shown in Listing 14.4.

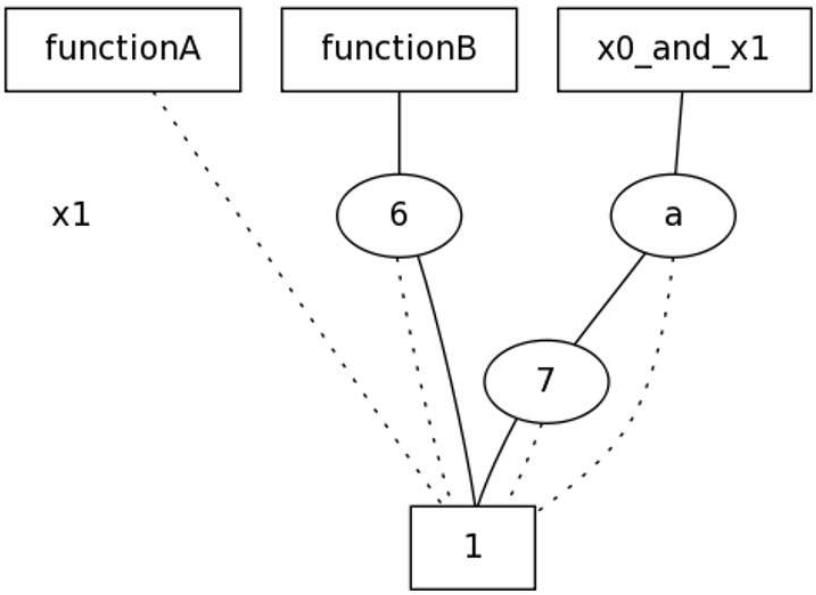


Fig. 14.2 BDD produced using CUDD library

The produced DOT file is shown for reference (and also as an example to writing DOT files, as explained in Section 19.4).

```
digraph "DD" {
size = "7.5,10"
center = true;
edge [dir = none];
{ node [shape = plaintext];
  edge [style = invis];
  "CONST NODES" [style = invis];
" x1 " -> "  " -> "CONST NODES"; }
{ rank = same; node [shape = box]; edge [style = invis];
"  functionA  " -> "  functionB  " -> "  x0_and_x1  "; }
{ rank = same; " x1 ";
"a";"6";}
{ rank = same; "  ";"7";}
{ rank = same; "CONST NODES";
{ node [shape = box]; "2";}}
"  functionA  " -> "2" [style = dotted];
"  functionB  " -> "6" [style = solid];
"  x0_and_x1  " -> "a" [style = solid];
"a" -> "7";
"a" -> "2" [style = dotted];
```

```
"6" -> "2";
"6" -> "2" [style = dotted];
"7" -> "2";
"7" -> "2" [style = dotted];
"2" [label = "1"];}
```

14.4 FWTools: Open Source GIS

FWTools include OpenEV, GDAL, MapServer, PROJ.4 and OGDI. These tools are described below. More details on Geographical Information Systems (GIS) is given in Section 14.9. OpenEV is a high-performance raster/vector desktop data viewer and analysis tool; an example is shown in Figure 14.3.



Fig. 14.3 OpenEV : Raster/Vector viewer

14.4.1 PROJ4

PROJ.4 is a cartographic projection library. PROJ4 is used by GIS projects (including GRASS, Section 14.9.1) to convert between coordinate systems. Listing 14.5 shows an example of coordinate conversion using PROJ4. It is also possible to perform conversion on the command line using the tools `proj` and `invproj`. Their usage is described below.

```
// \file proj4_example.cpp
// \author Sandeep Koranne, (C) 2010
// \description Example of using PROJ4
// for Cartographic Projection
#include <iostream>           // for program IO
#include <cassert>            // assertion checking
#include <cstdlib>            // exit
#include <proj_api.h>         // PROJ4 library

int main( int argc, char* argv[] ) {
  projPJ transformer, latitude_longitude;
  double gx, gy;

  transformer = pj_init_plus("+proj=utm +lon_0=112w +ellps=clrk66");
  if( !transformer ) {
    std::cerr << "Unable to construct PROJ..\n";
    exit(1);
  }
  latitude_longitude = pj_init_plus("+proj=latlong +ellps=clrk66");
  std::cout << "Enter value:";
  std::cin >> gx >> gy;
  pj_transform( latitude_longitude, transformer, 1, 1, &gx, &gy, NULL );
  std::cout << "Output : " << gx << "\t" << gy;
  std::cout << std::endl;
  return(0);
}
```

Listing 14.5 Example of cartographic projection using PROJ4

PROJ4 includes many projection systems including cylindrical, transverse Mercator, universal transverse Mercator (UTM), central cylinder projection, Airy projection and Miller projection.

The API of PROJ4 includes the `pj_transform` function:

```
int pj_transform( projPJ srcdefn,  // source projection format
                  projPJ dstdefn,  // destination projection format
                  long point_count, // number of points,
                  int point_offset, // which point in the data set,
                  double *x, double *y, // data input-output
                  double *z ); // for 3d transforms.
```

The API is initialized using a string which is the same as the `proj` filter as described below.

14.4.1.1 `proj`: forward cartographic projection filter

The command line tool `proj` implements the PROJ4 library as a command line program and can be used to perform coordinate conversions (e.g., UTM projections).

Consider the following example which calculates Boston's (approximately) UTM coordinates using a standard UTM central meridian of 112 degrees west. The command line option '-r' is used to *reverse* the default ordering of latitude-longitude coordinates.

```
$ proj -E +proj=utm +lon_0=112w +ellps=clrk66 -r << EOF
> 45 111.5W
> 75N -111
> EOF
45 111.5W 460591.19 4982854.80
75N -111 500000.00 8323452.59
```

In the above transcript we used the -E option which prints the input coordinates on the output as well. The related tool `geod` performs direct geodesic conversions. Geodesic (Great Circle) computations for determining latitude, longitude and back azimuth of a point given initial point latitude, longitude and azimuth and distance is supported. For example, to compute the azimuths and distance from Boston to Portland we can use:

```
$geod +ellps=clrk66 <<EOF -I +units=us-mi
             42d15N 71d07W 45d31N 123d41W
             EOF
-180d 0d 2907.509N 71d07W 45d31N 123d41W
```

To calculate Portland's location from Boston use:

```
$geod +ellps=clrk66 <<EOF +units=us-mi
             42d15N 71d07W -66d3150.141" 2587.504
             EOF
45d310.003"N              123d4059.985"W
75d3913.094"
```

The inverse projection tools are also available as `invproj` and `invgeod`.

14.4.2 GDAL : Geospatial Data Abstraction Library and OGR

GDAL is an open-source translator library and command-line utility tools for reading and writing a wide variety of geospatial raster (GDAL) and vector (OGR) formats. GDAL data model prescribes a GDAL Data Set as having a list of raster bands all corresponding to the same physical area, and having the same resolution. GDAL Data also has meta-data, a coordinate system and a geo-referencing transform. The size of the raster is included in the data itself. Using GDAL library we can write a C++ function to print the size of the dataset as:

```
#include <gdal_priv.h>
int WriteDataSetSize( const char* fileName ) {
  GDALDataset *pData;
  GDALAllRegister();
```

```
  pData = (GDALDataset*) GDALOpen( fileName, GA_ReadOnly );
  if( pData == NULL ) {
    std::cerr << ``Unable to open file : `` << fileName ;
    exit(1);
  }
  std::cout << ``Raster X Size = `` << pData->GetRasterXSize()
            << ``\n Raster Y Size = `` << pData->GetRasterYSize()
            << ``\n Raster Count = `` << pData->GetRasterCount()
            << std::endl;
}
```

Listing 14.6 Example of using GDAL library

The GDAL library provides access to the raster band data, one band at a time using the `GDALRasterBand` object. Related tools include OGDI, which is a multi-format raster and vector reading technology, and MapServer which is a map server using Web CGI and GeoCode is a geocoding library.

14.5 GNU Image Manipulation Program

GNU Image Manipulation Program (GIMP) is an image processing and paint program which is able to process a number of graphic file formats. GIMP is used to edit and manipulate bitmap images and can be used to convert between formats. It is also possible to use GIMP as a paint program as it features drawing tools such as brush, pencil and clone. An example of an image opened in GIMP for processing is shown in Figure 14.4.

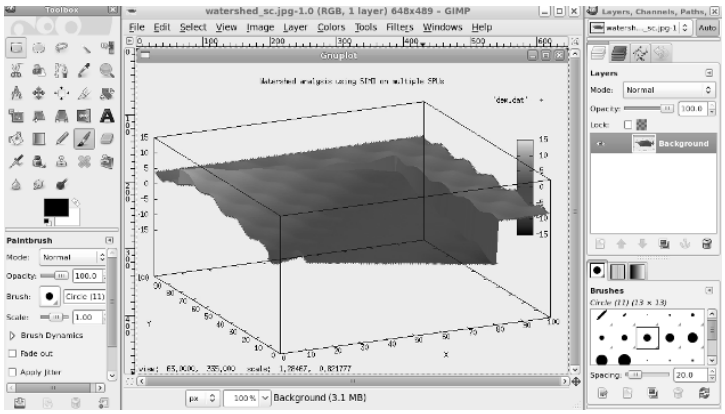
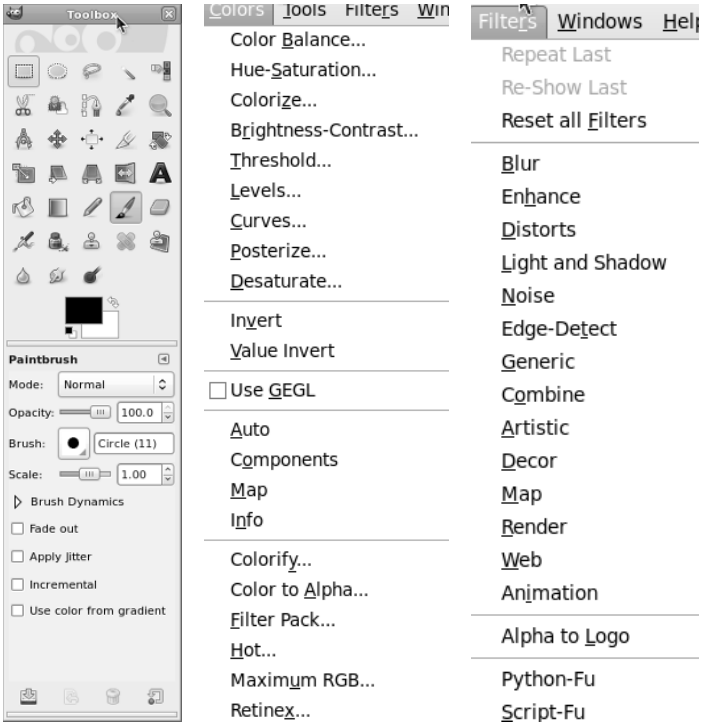


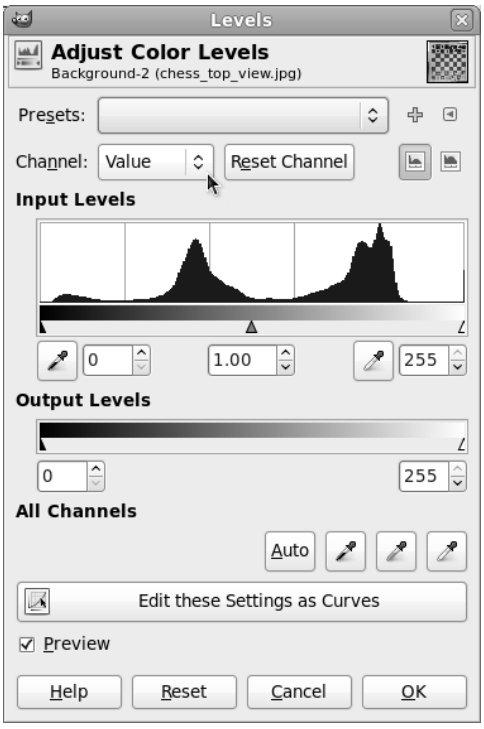
Fig. 14.4 GNU Image Manipulation Program (GIMP)

GIMP has a number of tools built into it, as shown in Figure 14.5.

GIMP also supports *plugins* which provide additional functionality. A plugin can access the bitmap contents of the image currently loaded into the GIMP program and perform analysis, and processing on it. If the image contents are modified, GIMP can then write the resulting output to disk in any supported file format.



(a) GIMP Tools (b) Color (c) Filters



(d) Color levels

Fig. 14.5 GIMP built in tools and filters

14.6 Computational Fluid Dynamics using OpenFOAM

OpenFOAM is an acronym for Open Field Operation and Manipulation. OpenFOAM is an open-source computational fluid dynamics package and it can support CFD applications on complex flows involving chemical reactions turbulence and heat transfers. It has a plugin to ParaView (see Section 19.17), which is used to view the CFD geometry and perform analysis using the GUI.

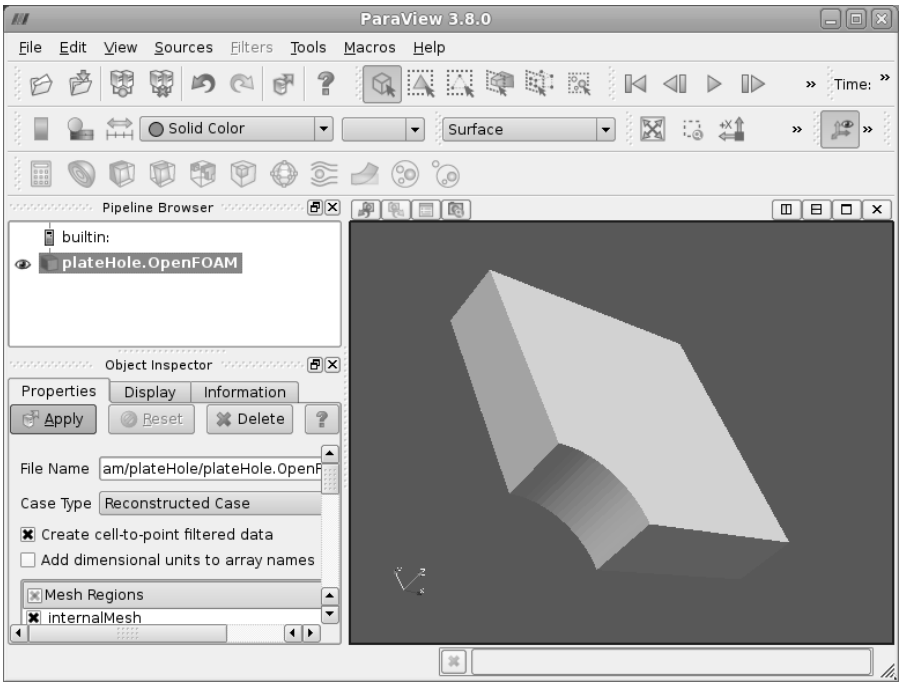


Fig. 14.6 ParaView as a frontend to OpenFOAM

The geometry for analysis can be defined as an ASCII file:

```
FoamFile
{
    version     2.0;
    format      ascii;
    class       dictionary;
    object      blockMeshDict;
}

convertToMeters 1;

vertices
(
    (0.5 0 0)
    (1 0 0)
    (2 0 0)
    (2 0.707107 0)
```

```
    (0.707107 0.707107 0)
...
    (0 2 0.5)
    (0 1 0.5)
    (0 0.5 0.5)
);

blocks
(
    hex (5 4 9 10 16 15 20 21) (10 10 1)
       simpleGrading (1 1 1)
    hex (0 1 4 5 11 12 15 16) (10 10 1)
       simpleGrading (1 1 1)
    hex (1 2 3 4 12 13 14 15) (20 10 1)
       simpleGrading (1 1 1)
    hex (4 3 6 7 15 14 17 18) (20 20 1)
       simpleGrading (1 1 1)
    hex (9 4 7 8 20 15 18 19) (10 20 1)
       simpleGrading (1 1 1)
);

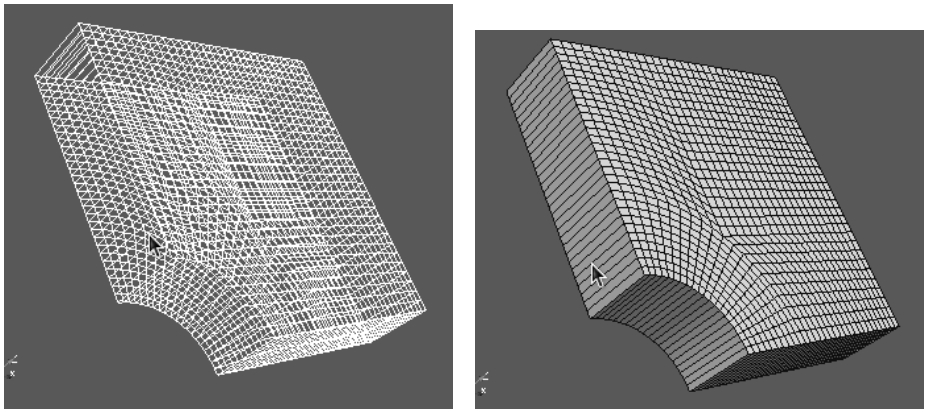
edges
(
    arc 0 5 (0.469846 0.17101 0)
    arc 5 10 (0.17101 0.469846 0)
    arc 1 4 (0.939693 0.34202 0)
    arc 4 9 (0.34202 0.939693 0)
    arc 11 16 (0.469846 0.17101 0.5)
    arc 16 21 (0.17101 0.469846 0.5)
    arc 12 15 (0.939693 0.34202 0.5)
    arc 15 20 (0.34202 0.939693 0.5)
);
```

Figure 14.6 shows the geometry of the plate with a corner cut off. OpenFOAM has a mesh generator which can be invoked with the `blockMesh` command; this command converts the geometry into *mesh* form. The size of the mesh can be specified.

Once the design has been meshed we can perform analysis using a number of tools provided with OpenFOAM. OpenFOAM by itself is a collection of C++ libraries, but several applications come pre-built with the OpenFOAM release. The applications can be divided into two categories:

1. solvers: each application is designed to solve a very specific problem in CFD,
2. utilities: pre-processors, format converters, data manipulation and post-processing tools.

OpenFOAM allows (and indeed is designed) for runtime linking of user-defined code to provide either new solvers, or new data representations. The shared libraries (which have to be compiled using the OpenFOAM header files) can be placed in the 'controlDict' file for a case using the 'libs' keyword. Then at runtime OpenFOAM will automatically load the shared libraries into the analysis tool for that case.



(a) OpenFOAM mesh (b) OpenFOAM surface with edges

Fig. 14.7 OpenFOAM block mesh

14.6.0.1 Case and File Structure in OpenFOAM

The basic directory structure for an example is shown below:

```
$ ls
0  0.1  0.2  0.3  0.4  0.5  cavity.OpenFOAM
constant  system
$ ls -l system/
total 12
  controlDict
  fvSchemes
  fvSolution

$ ls -l constant/
total 8
 polyMesh
 transportProperties
[skoranne@celex cavity]$ ls -l constant/polyMesh/
total 68
 blockMeshDict
 boundary
 faces
 neighbour
 owner
 points
```

The ‘system’ directory is used for setting parameters associated with the solution procedure. The ‘constant’ directory contains a description of the case mesh in the ‘polyMesh’ sub-directory (as shown above). The physical properties for the appli-

cation are located in files such as 'transportProperties'. Data in OpenFOAM files is stored as Key-Value dictionaries.

14.6.0.2 CFD Solvers in OpenFOAM

OpenFOAM contains solvers for incompressible flow, solid body stress analysis and combustion analysis. In addition the conventional CFD solvers such as Laplacian solver, scalar transport and laminar flow are also included. In-fact, OpenFOAM also includes a solver for Black-Scholes option pricing (the `financialFoam` solver).

14.6.0.3 Utilities in OpenFOAM

The utilities in OpenFOAM can be classified into the following categories:

1. Pre-processing: box-drawing, initialization of data into OpenFOAM case directories, volume field mapping, and molecular dynamics,
2. Mesh generation and conversion: in the example above we saw the use of `blockMesh` for mesh generation. OpenFOAM also has utilities for converting formats from other CFD packages to the OpenFOAM format,
3. post-processing: these utilities include graphical display and rendering as well as data export from OpenFOAM format to other CFD packages,
4. Sampling Utilities: OpenFOAM `sample` command can be used to probe locations and cells in the case data for a design in OpenFOAM. It can be used to sample field data and generate interpolation data points which are amenable for plotting and external analysis.

14.7 Molecular Dynamics

Computational Molecular Dynamics has become a major driver of high-performance computing. Computer simulations of molecules are used in solving problems in physical chemistry, structure determination in crystallography, and in experimental drug discovery. The latter has significant potential to improve the current treatment options for many of the common ailments, and thus is of considerable social and financial value. In most of the above application domains, a collection of physical structures (atoms, molecules, proteins, etc) are suspended in a fluid of known properties. Starting with a known state (or preconditioned state), the goal of the computer simulation is to calculate the trajectory (physical motion) of the structure's components over a *time step*. Integrating the motion of individual components over a long enough time period can be used to gather information about the physical properties of the compound under study. In this section we describe some of the common molecular dynamics codes and applications.

14.7.1 NAMD

NAMD was designed to support parallel execution on a networked cluster and thus is able to harness the power of a Beowulf class PC-clusters. NAMD supports the following analysis:

1. PME (Particle Mesh Ewald): NAMD implements the PME algorithm for full electrostatic interaction computation. This algorithm reduces the time complexity of the computation to $O(n \log n)$ from $O(n^2)$, which provides significant speedup,
2. Force field compatibility: the force field used in NAMD is the same as those used by other molecular dynamics programs which can help in correlating results, as as well migration of simulations from one software to another,
3. Multiple time stepping: NAMD used Verlet integration method to advance the time step,
4. Interactive simulations
5. Load Balancing.

14.7.2 GROMACS

GROMACS is an open-source package to perform molecular dynamics. Its main features are:

1. Efficiency: GROMACS code base has been systematically optimized for high-performance using latest compilers, and CPU instructions, including 64-bit registers and SSE2,
2. User friendly: GROMACS topology files are plain ASCII text files which are simple to generate and edit,
3. Writing trajectory data using lossy compression :accuracy of compression is user selectable, but provides significant compaction,
4. MPI based parallel programming

14.7.3 Molecular Visualization

Molecules can be viewed using JMol or RasMol, both open-source packages for molecular visualization.

14.7.3.1 JMol

JMol is an open-source molecule viewer which comprises of:

1. JMolApplet: web browser applet which can be integrated into web pages,

2. JMol Application: is a standalone Java application,
3. JMolViewer: is a development tool that can be integrated into other Java programs.

An example of JMol application viewing a PDB file is shown in Figure 14.8.

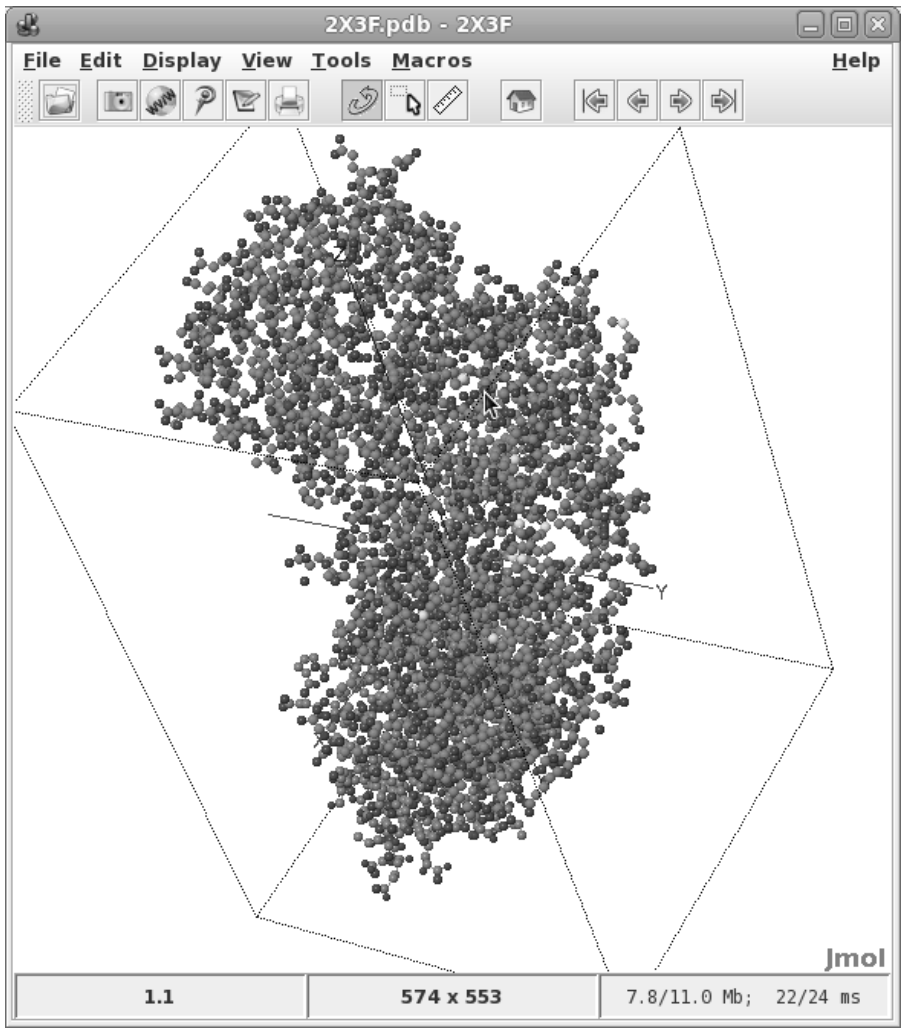


Fig. 14.8 JMol : Molecule Viewer application

14.7.4 Foldng@Home

No section on open-source molecular dynamics can be complete without a reference to Folding@Home. Folding@Home is a distributed computing project, which harnesses the power of idle desktop personal computers from all over the world ti run

software to perform protein folding. Work units are distributed to each of the computers, each unit represents some folding computation problem which needs to be solved. Collectively, Folding@Home, represents one of the largest supercomputers implemented using distributed computing. Folding@Home clients are available for a number of operating systems, computers and even supercomputers.

14.8 Audacity

Audacity is a graphical audio editor. It uses the `libsndfile` library to read/write many uncompressed file formats. An example of a WAV file being edited is shown in Figure 14.9. It includes support for filters such as noise removal and stereo editing. Batch processing applications use `sox` which is described in Section 14.8.1.

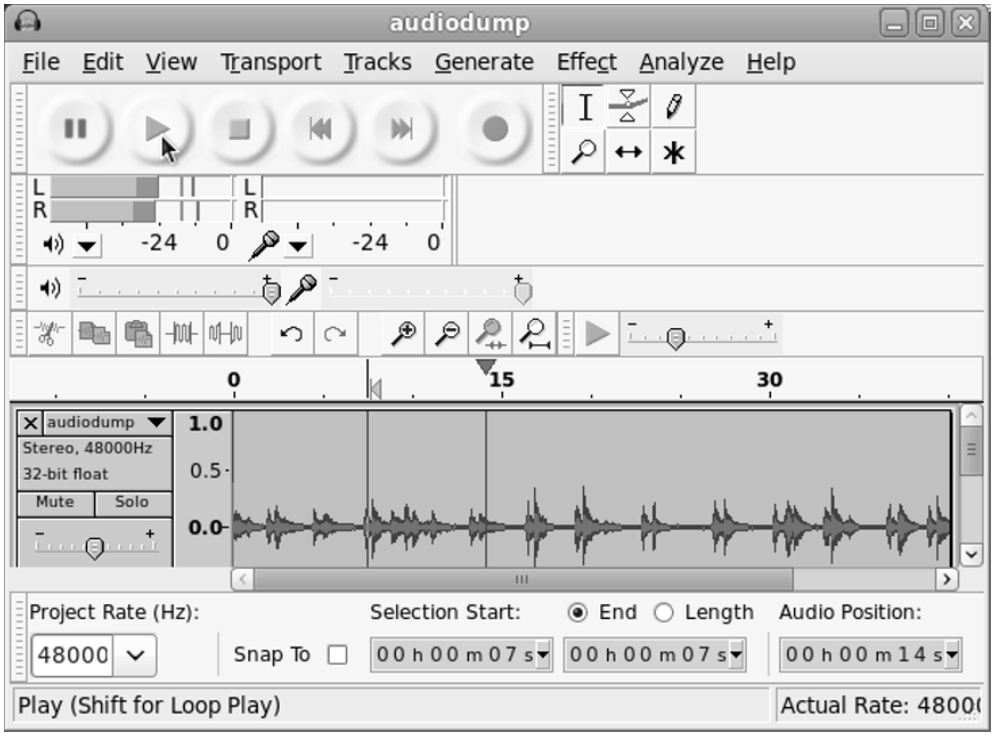


Fig. 14.9 Audacity : audio editor

14.8.1 Sound Exchange : sox

Sound Exchange (sox) is a computer program that can read/write audio files and perform audio processing on them. It can combine various input file, synthesize

sounds, and even act as a simple audio player. The `sox` processing chain can be described as:

```
Input -> Combiner -> Effects -> Output
```

The input and output phases can perform conversion to/from a wide variety of file formats. The combiner and effects can include (i) sampling rate change, (ii) sampling size change, (iii) mono down mix, (iv) volume and dither control, and (v) speed control (to speedup or slow down the audio). `sox` can also act as a sound recorder with auto detection of sound.

14.9 Geographical Information Systems

With the advent of localization based services and the widespread use of information technology in urban planning, the role of geographical information systems (GIS) software has burgeoned. In the open-source community the GRASS GIS system, and Quantum GIS (qgis) are popular, and we describe these below.

14.9.1 GRASS GIS

GRASS is an open-source geographical information system (GIS) which can handle raster, topological vector, image processing, and graphic data. An example of using GRASS GIS is shown in Figure 14.10 which shows county data in the USA.

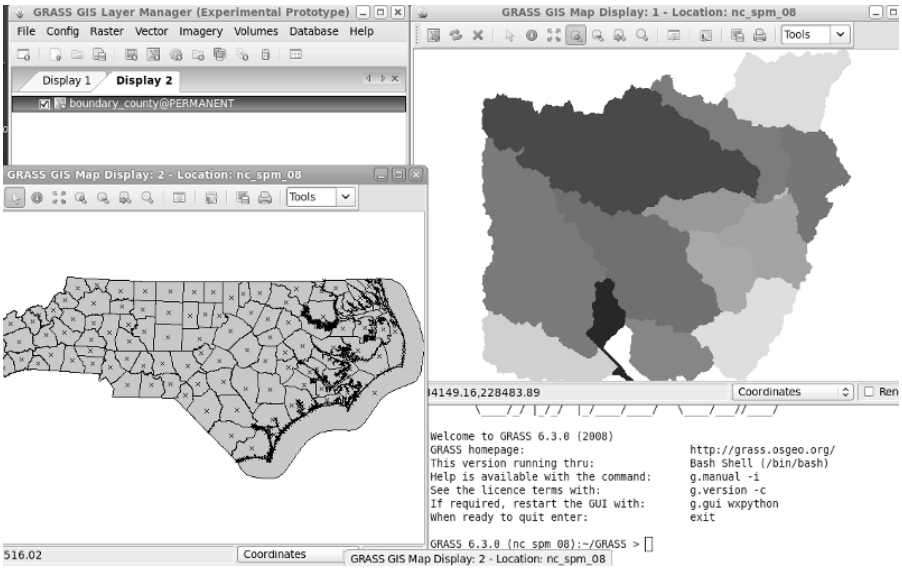


Fig. 14.10 GRASS GIS (geographical information system)

Recently a new topological 2D/3D vector engine has been added to GRASS (version 6.0) which adds support for vector network analysis. Moreover, GRASS can store attributes in SQL-based databases such as MySQL (see Section 20.6), PostgreSQL/PostGIS (see Section 20.5) and SQLite (see Section 20.7).

14.9.2 Quantum GIS

Quantum GIS (QGIS) is another open-source GIS software. QGIS is written in C++ and uses the Qt libraries (see Section 19.1.3), and as such is very portable and runs on GNU/Linux as well as Mac OS X, and Microsoft Windows. An example of QGIS is shown in Figure 14.11. Map data from USA atlas is available at `http://nationalatlas.gov` and we have used the county map, location of dams, and road network in the examples below. The county area of Portland, Oregon is shown in Figure 14.11.

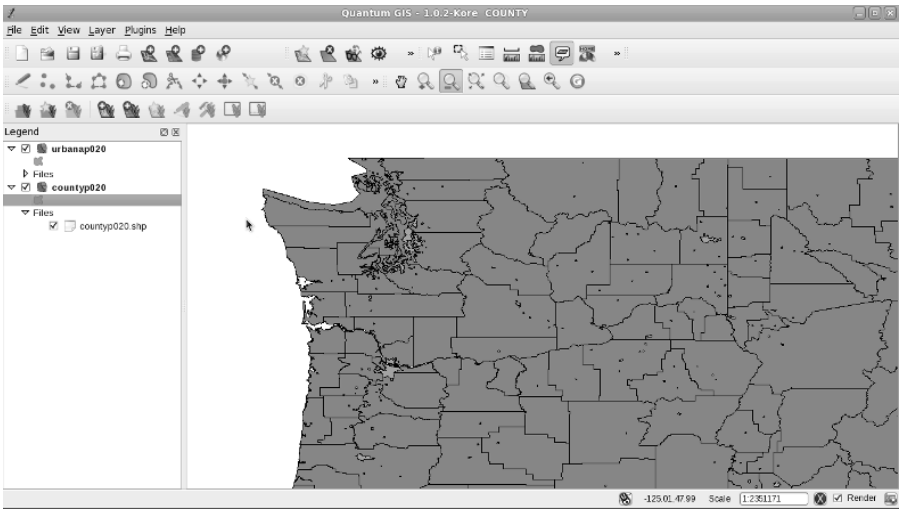


Fig. 14.11 Quantum GIS (geographical information system)

We add another vector shape file, the location of dams, and urban areas. We see that there are a number of dams located near the urban areas as shown in Figure 14.12.

A common use of GIS is in road network planning, and transportation planning. The road network near Portland is loaded from the data file and can be overlayed on the existing maps, as shown in Figure 14.13.

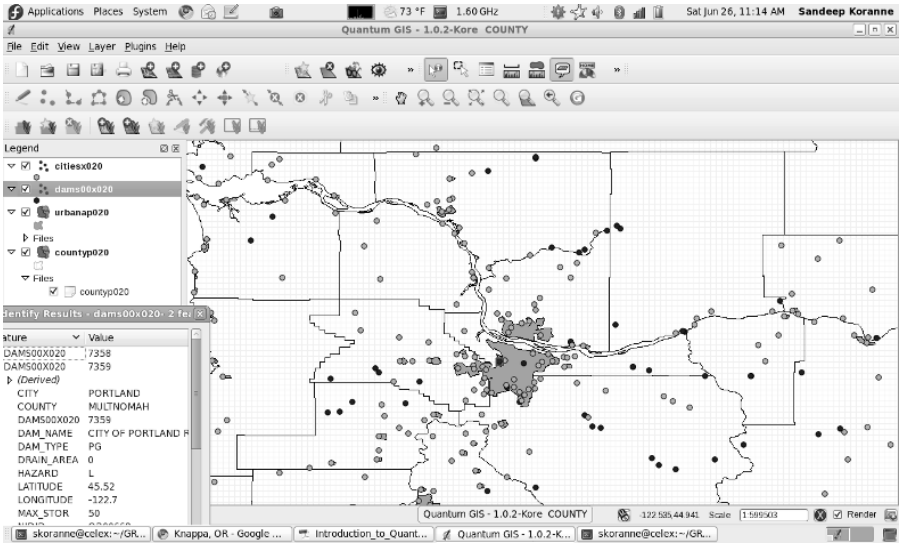


Fig. 14.12 Portland and surrounding area, urban area, county and dams.

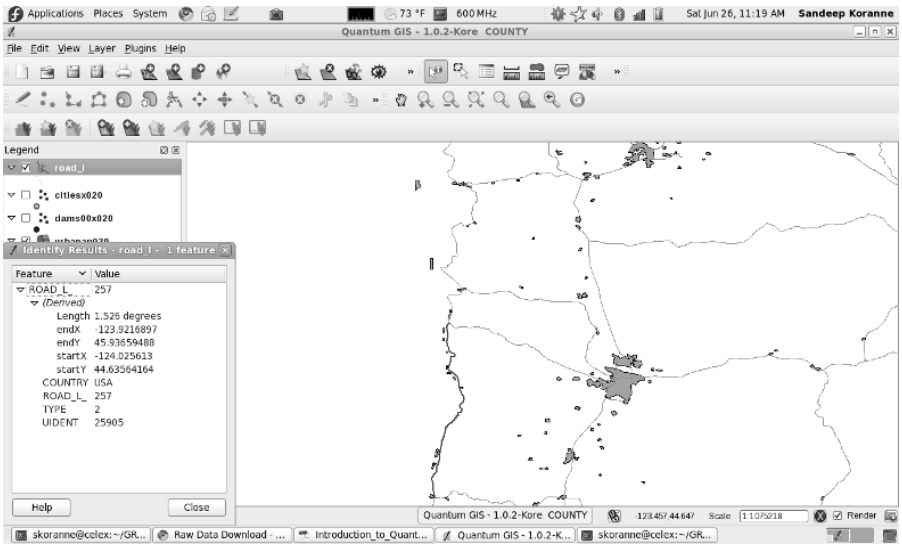


Fig. 14.13 Road network, vector shape file

14.10 QCAD : 2d CAD Tools

Computer Aided Design is the discipline of using computer and information processing to aid in mechanical design. There is a fundamental difference between a CAD program and a general drawing program as shown in Section 19.7, although it is possible to use a general drawing program for initial schematics, the CAD program provides additional features such as ISO compliant dimensioning, exact measurement, wide variety of grid snapping and more. In this section we describe

QCad an open-source CAD program for two-dimensional drafting. An example of QCad drawing is shown in Figure 14.14.

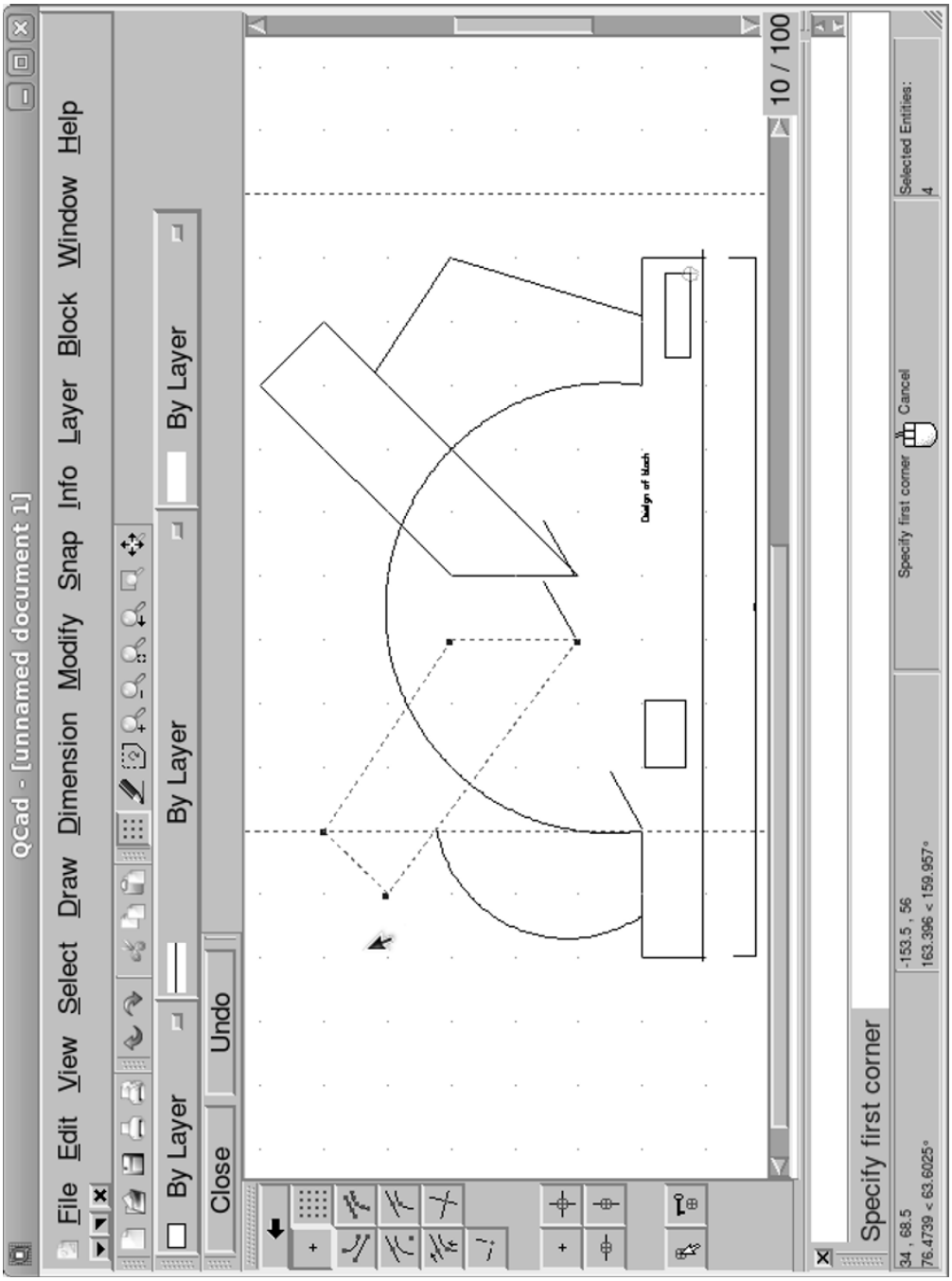


Fig. 14.14 QCad: Computer Aided Design

14.11 BRL-CAD

BRL-CAD is a constructive solid geometry package designed for a wide variety of military and industrial purposes. The software is open-source and as of version 7.16B contains a number of tools including (a) interactive geometry editor, (b) ray-tracing, and (c) network distributed image processing.

The interactive geometry editor is termed the Multi-Device Geometry Editor (`mged`).

14.11.0.1 Multi-Device Geometry Editor

The editor can be launched using the `mged` command line tool. An existing geometry database can be specified on the command-line or opened using the GUI. The GUI has an interactive shell window which accepts text commands, while the GUI menus provide the same functionality graphically.

Constructive Solid Geometry (CSG), as the name implies, is the process of model construction through the use and combination of primitive objects such as (a) ellipsoid (generalized sphere), (b) right parallelepiped, (c) toric shapes, and more. These primitives can be placed in the model at appropriate coordinates, scaled and transformed using the editor to form the desired model. The *center-of-view*, scaling and zooming, azimuth view, of the current display can be controlled by the user. Consider an example shown in Figure 14.15.

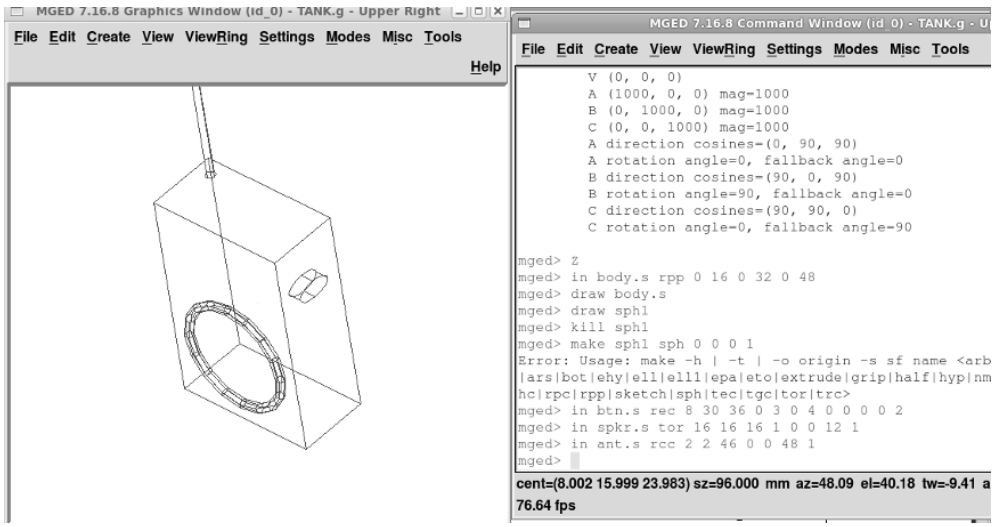


Fig. 14.15 BRL-CAD mged Geometry Editor

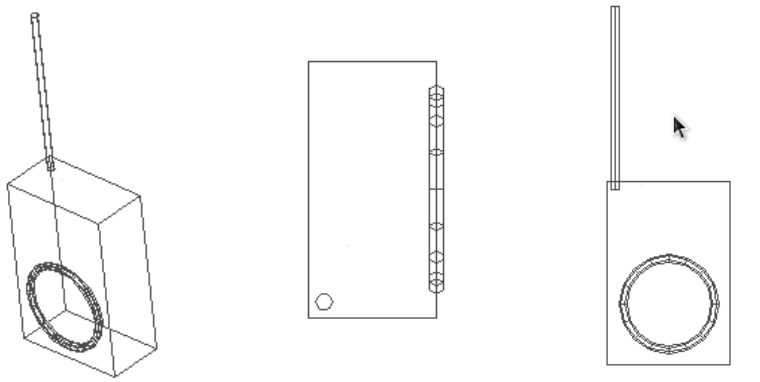
The construction of objects can be carried out by entering commands in the console:

```
in rcc3s rcc 3 2 4 0.2 0.2 0.3 0.4
```

```
in rcc4s rcc -3 2 4 0.2 0.2 5.0 0.4
mged> in rcc5s
Enter solid type: rcc
Enter X, Y, Z of vertex: 3 3 4
Enter X, Y, Z of height (H) vector: 0 0 1
Enter radius: 0.4
```

This command `in`, inserts a right circular cylinder at the given (x, y, z) coordinates as the RCC's center. The height of the RCC is entered next in the current unit, followed by the radius of the cylinder.

Since BRL-CAD is used to order parts for machining directly from the model, the engineering drawing views of the model also need to be generated as shown in Figure 14.16.



(a) Perspective view (b) Top view (c) Front view

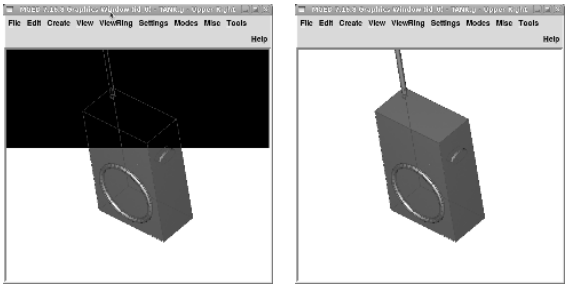
Fig. 14.16 Perspective view, top and front view

A realistic image of the constructed model can be generated using the *raytracer* which is included with the software. An example of the raytracer in action and the produced final image is shown in Figure 14.17(a) and (b).

Moreover, BRL-CAD also supports performing Boolean operation on the solid volumes. The Boolean can be *union*, *subtraction* and *intersection*. Consider a machine part comprised of a cylinder in which cutouts for rivet-heads have to be drilled as shown in Figure 14.18(a). The part itself comprises of a RCC, on top of which we add 4 spheres and a small RCC. The content of the current file can be listed using the `ls` command; while the hierarchical constituents of a model can be analyzed using the `tree` command. The Boolean operation can be performed as:

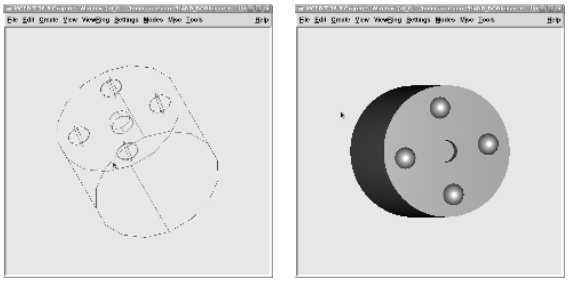
```
r mp1.part u rcc1 - sph1 - sph2 - sph3 - sph4 - rcc2
```

The final rendering of the machine part is shown in Figure 14.18(b). The rendering can be influenced by choosing a *material* to apply for the object. The transparency and reflectance of the surface can be changed using this property. Figures



(a) Partial raytracer (b) Raytraced image

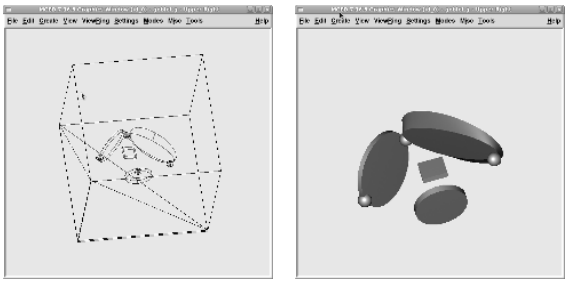
Fig. 14.17 Raytracer in BRL-CAD



(a) Machine part (b) Final rendering

Fig. 14.18 Example machine part in BRL-CAD

can be rotated in the editor using the 'Z' command, and using the arrow keys various azimuth and elevation angle views can be generated. Multiple parts of the drawing can be constructed separately and assembled later using the `comb` function and raytraced together.



(a) Another machine part (b) Final rendering

Fig. 14.19 Example machine part in BRL-CAD

An object can be copied to another using the `cp` command in the console. Attributes of objects such as scale, translation and individual object property such

as *radius*, *height* can also be changed. Another 3d geometry construction software named *Blender* is discussed in Section 14.12. A dedicated raytracing program (POVray) is discussed in Section 19.9.

14.12 Blender

Blender is an open-source software to create 2d and 3d content aimed at media professionals and artists. An example of Blender is shown in Figure 14.20 and Figure 14.21.

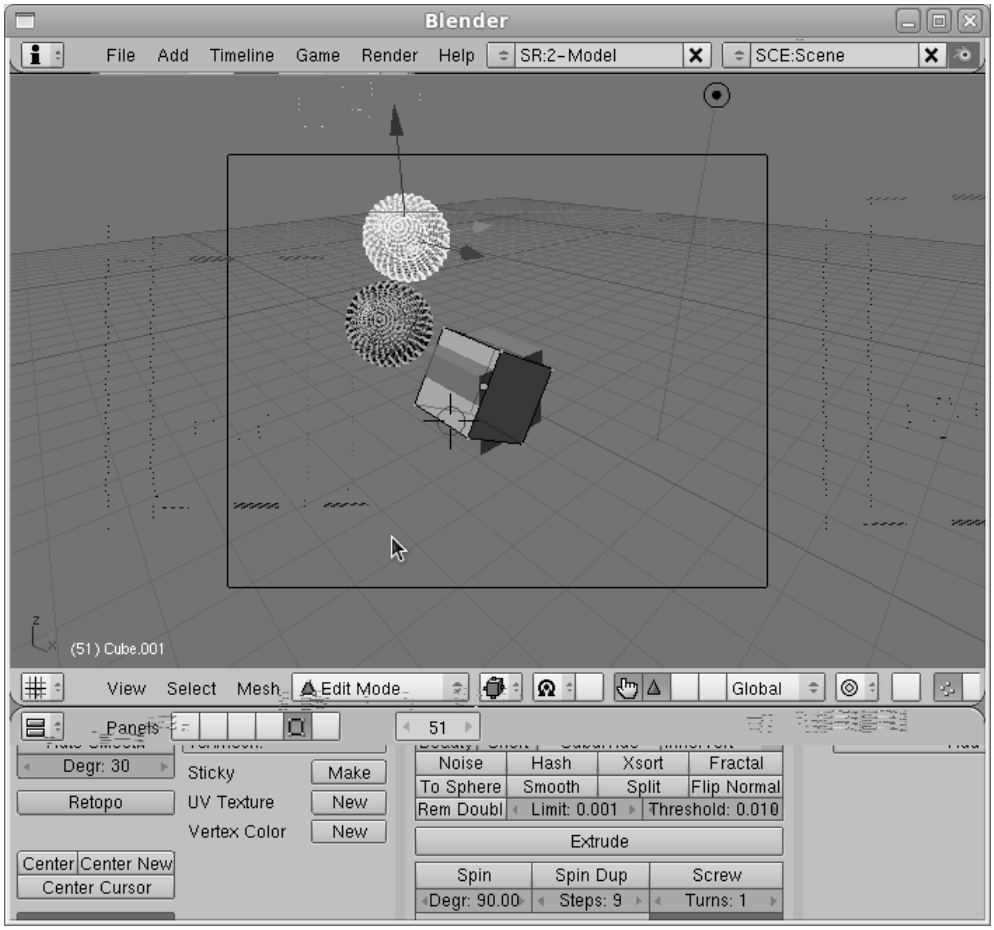
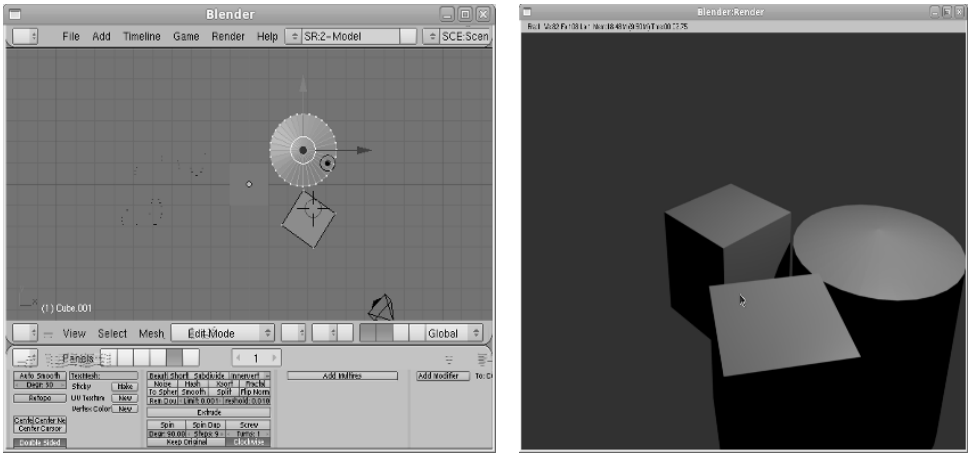


Fig. 14.20 Main GUI screen of Blender

The key features of Blender include:

1. Tools for content creation: including modeling, UV-mapping, texturing, rigging, animation, rendering, compositing, post-production and game creation,
2. Cross platform availability,

3. Small executable size.



(a) A cylinder in Blender (b) Raytraced rendering

Fig. 14.21 Rendering in Blender

Blender has its unique GUI with many actions tied to single key shortcuts which increases the productivity of the designer, but has a learning curve.

14.13 Conclusion

In this chapter we presented engineering libraries such as Computer Vision, CImg and FWTools. Geospatial data abstractions are becoming very important with the rise of location aware computing, and several open-source tools such as GDAL and PROJ4 were described in this chapter. Image processing, audio processing, and computational fluid dynamics (CFD) that are part of many engineering were presented. Molecular dynamics simulators and viewers were also discussed. Information Systems (GIS) including GRASS and QGIS are described in Section 14.9.1. Mechanical engineering, as well as use of mechanical CAD software in other disciplines can be accomplished using open-source tools such as QCAD. Solid modeling tools BRL-CAD and Blender were described in this chapter.